INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

Distributed Systems

Message Passing Systems

Recap

- Algorithmic challenges in distributed computing
- Buffered / unbuffered, Synchronous / Asysnchronous. Blocking / non-blocking send and recv
- MPI / MPICH
- MPI functions

Algorithmic challenges in distributed computing

- **Load balancing**
  - Data / Computation migration
  - Distributed Scheduling
- **Real-time scheduling**
  - Challenging in a distributed system where a global view of the system state is absent
  - message propagation delays can't be controlled or predicted, which makes meeting real-time guarantees
- **Performance**
  - Improving performance
  - Measuring performance (Metrics)

Example 2

```
if (rank == 0) {
    send_data = 100;
    printf("Process 0 sending %d to process 1\n", send_data);
    MPI_Send(&send_data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    MPI_Recv(&recv_data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &status);
    printf("Process 0 received %d from process 1\n", recv_data);
  } else {
    // Recv from process 0 first to avoid potential deadlock in unbuffered mode
    MPI_Recv(&recv_data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
    printf("Process 1 received %d from process 0\n", recv_data);
    send_data = 200;
    printf("Process 1 sending %d to process 0\n", send_data);
    MPI_Send(&send_data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
  }
```

Modifications

Modification 1

If for rank 1 process first MPI_Send is called and then MPI_Recv is called?

Modification 2

If for both processes MPI_Send is replaced by MPI_Ssend in the program with modification 1?

Example (Deadlock)

```
if (rank == 0) {
      send_data = 100;
      printf("Process 0 sending %d to process 1\n", send_data);
      MPI_Send(&send_data, 1, MPI_INT, 1, 1, MPI_COMM_WORLD);
      MPI_Recv(&recv_data, 1, MPI_INT, 1, 1, MPI_COMM_WORLD, &status);
      printf("Process 0 received %d from process 1\n", recv_data);
  } else {
      MPI_Recv(&recv_data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
      printf("Process 1 received %d from process 0\n", recv_data);
      send_data = 200;
      printf("Process 1 sending %d to process 0\n", send_data);
      MPI_Send(&send_data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
  }
```

MPI_Isend

- This function is Non-blocking and returns immediately
- The caller gets a MPI_Request handle
- Caller must not modify or deallocate the send buffer until the communication operation is confirmed as complete
- This may be checked by calling a completion routine like MPI_Wait or MPI_Test

MPI_Bsend (Buffered Send)

- **Local Operation:** It completes without needing a matching receive from the destination process.
- **Buffering Mechanism**: The user must provide a buffer using MPI_Buffer_attach before calling MPI_Bsend.
- **Safety & Completion**: When MPI_Bsend returns, the send buffer can be safely reused.

It eliminates synchronization overhead but requires manual buffer management to avoid overflow.

MPI_Bsend (Buffered Send)

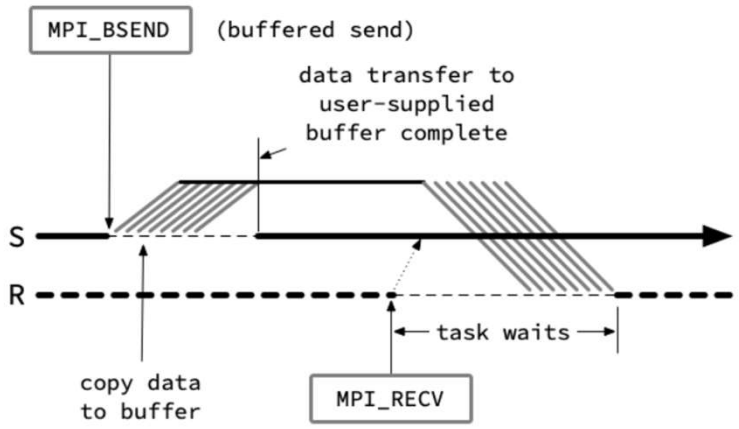


Diagram of a blocking buffered send.

Socket API

TCP Echo Server

```
int main(int argc, char **argv)
{       ...
        listenfd = socket(AF_INET, SOCK_STREAM, 0);

        bzero(&servaddr, sizeof(servaddr));
        servaddr.sin_family      = AF_INET;
        servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
        servaddr.sin_port        = htons(SERV_PORT);

        bind(listenfd, (struct sockaddr *) &servaddr, sizeof(servaddr));
        listen(listenfd, 5);

        for ( ; ; ) {
          clen = sizeof(cliaddr);
          connfd = accept(listenfd, (struct sockaddr*) &cliaddr, &cllen);

          if ( (childpid = fork()) == 0) {    /* child process */
               close(listenfd);       /* close listening socket */
               n = recv(connfd, line, 512, 0);
               send(connfd, line, n, 0);
               close(connfd);
               exit(0);
               }
               close(connfd);
          }
}
```

TCP Echo Client

```
int main(int argc, char **argv)
{
        int                     sockfd;
        struct sockaddr_in      servaddr;

        if (argc != 2)
                err_quit("usage: tcpcli <IPaddress>");

        sockfd = socket(AF_INET, SOCK_STREAM, 0);

        bzero(&servaddr, sizeof(servaddr));
        servaddr.sin_family = AF_INET;
        servaddr.sin_addr.s_addr = inet_addr(SERV_HOST_ADDR);
        servaddr.sin_port = htons(SERV_PORT);

        connect(sockfd, (struct sockaddr*) &servaddr, sizeof(servaddr));

        fgets(sendline, 512, stdin);          /* do it all */
        n = strlen(sendline);
        send(sockfd, sendline, n, 0);
        n = recv(sockfd, recvline, 512, 0);
        if (n < 0) ...
        recvline[n] = 0;
        fputs(recvline, stdout);
        exit(0);
}
```

Socket API (TCP)

Case 1

- Add the following lines in server code before recv():
  sleep(10);
  printf("calling recv now\n");

  Add the following lines after send in client's code:
  printf("message transmitted\n");

Case 2

- Add MSG_DONTWAIT flag in call to recv() (change read to recv) function in server code. Run server and then client. Do not type any string when client program asks for it.
  - Print contents of buffer in server. What does it print?
  - Type a line in client. What is printed by the client?

Socket API (TCP)

- Client wants to be sure that destination TCP has received the data
- Client wants to be sure that destination user process has received the data

SO_LINGER Option

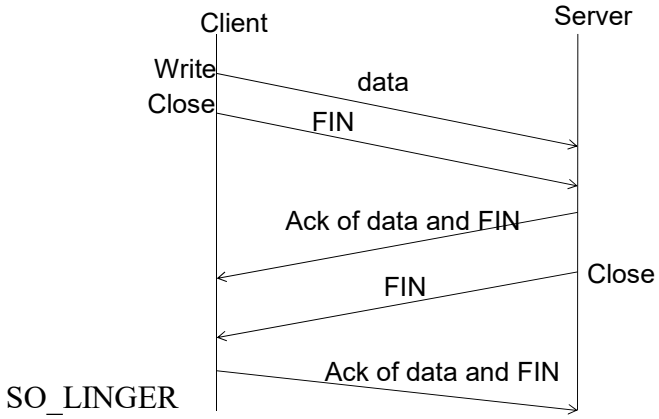
- Specifies how the close function operates for a connection-oriented protocol
  - **default** - close returns immediately but the system tries to deliver the remaining data in the socket send buffer
  - this option uses the following structure

```
struct linger {
        int l_onoff ;
        int l_linger ;
};
```

  - l_onoff = 0
  - l_onoff is nonzero and l_linger = 0 : TCP aborts the connection when it is closed (sends RST if descriptor reference count is zero)
  - l_onoff and l_linger both are nonzero : if there is any data still remaining, the process is put to sleep until either

    (a) all data is sent and ack and FIN is ack by the peer

    (b) the linger time expires



- process should check the return value of close.
- If client wants to know that the server process has read the data
  - call shutdown

Processor synchrony

Synchronous versus asynchronous processors:

- *Processor synchrony* indicates that all the processors execute in lock-step with their clocks synchronized.

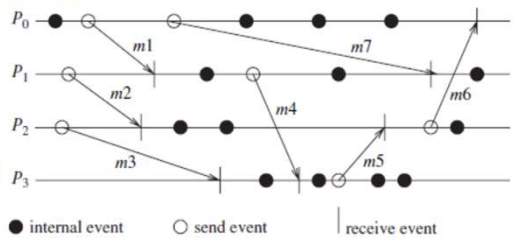
  - not attainable in a distributed system

- The processors are generally synchronized for a large granularity of code, usually termed as a *step*
- How this abstraction is implemented?

  use some form of barrier synchronization to ensure that no processor begins executing the next step of code until all the processors have completed executing the previous steps of code.

Synchronous / Asynchronous executions

- An *asynchronous execution* is an execution in which
  i. there is no processor synchrony and there is no bound on the drift rate of processor clocks
  ii. message delays (transmission + propagation times) are finite but unbounded, and
  iii. there is no upper bound on the time taken by a process to execute a step.



Asynchronous execution

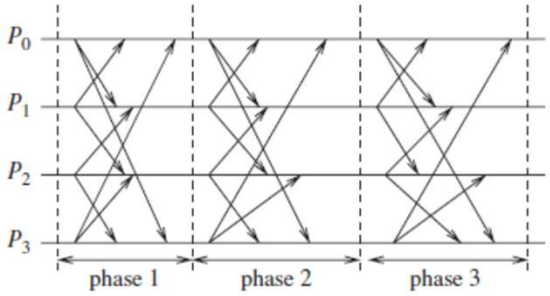
Synchronous / Asynchronous executions

- A *synchronous execution* is an execution in which

i. processors are synchronized and the clock drift rate between any two processors is bounded

ii. message delivery (transmission + delivery) times are such that they occur in one logical step or round, and

iii. there is a known upper bound on the time taken by a process to execute a step.



Designing Distributed Algorithms

Synchronous executions

- It is easier to design and verify algorithms assuming synchronous executions
- However, it is practically difficult to build a completely synchronous system
- Simulate synchrony – this may involve delaying or blocking some processes for some time durations.
- Synchronous execution is an abstraction that needs to be provided to the programs.
- Fewer the steps or "synchronizations" of the processors, the lower the delays and costs.

Virtual synchrony

- Virtual Synchrony is a mechanism to tackle the **atomic multicast problem**.
- It gives us a model for managing a group of (replicated) processes and coordinating communication with that group.
- With virtual synchrony, **processes can join or leave** (or be evicted) a group

Virtual synchrony

- Virtual synchrony software implements an atomic multicast to the group.

  => message is either delivered to *all* processes in the group or to *none*.

  => Group membership change events are ordered together with ordinary messages.

- Message ordering requirements can often be specified by the programmer but we nominally expect causal ordering of any messages

Virtual synchrony

Group members may receive any of three types of events:

- **Regular message**. This message is an input to the program (the state machine), although its delivery to the application may be delayed based on message ordering policy and our ability to be sure that other group members have received the message.
- **View change**. A message that informs the process of a group membership change. This affects any multicasts that are generated by the process from this point forward.
- **Checkpoint request**.
  - When a process joins a group, it needs to bring its state up to date so that it contains the latest version.
  - Send a *checkpoint request* message to any other process in the group. That process will send its current state to the requesting process.

IIT ROORKEE
23

View changes

Scenario:

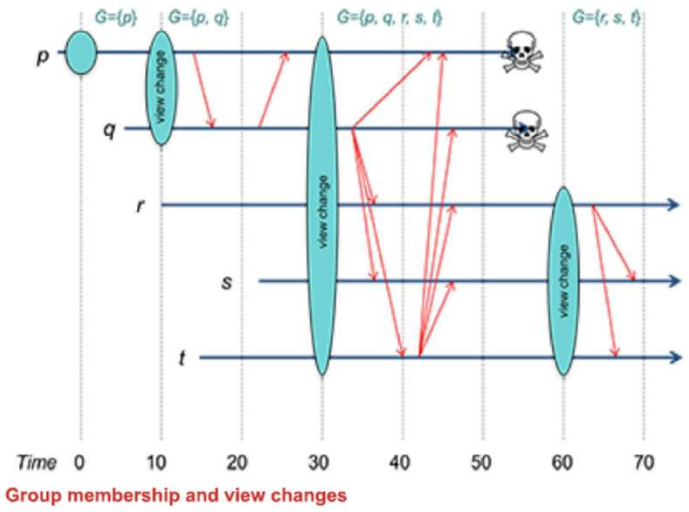
- We have a view change because a new process is joining the group.

  &

- At the same time, we have some regular messages being multicast to the group.
  - We cannot allow the condition where some messages may be delivered to the old group and some other messages may be delivered to the new group because that sender already received the *view change* message.
- We need to guarantee atomicity: *all or nothing* behavior
- ISIS toolkit.

IIT ROORKEE
24

Virtual synchrony



Group membership and view changes

Mutual Exclusion (Recap)

- Lamport's Algorithm
- Ricart-Agarwala Algorithm

Maekawa's Algorithm

- A site does not require permission from every other site
- Message complexity
  - Lamport's Algorithm
  - Ricart-Agarwala Algorithm
  - Thomas Algorithm (majority consensus rule)
- Uses only c * sqrt(N) messages to create mutual exclusion
  - N is the number of nodes and
  - c a constant between 3 and 5

Quorum based mutual exclusion

- Permission is obtained from only a subset of other processes, called the Request Set (or Quorum)
- Separate Request Set S_i for each process i
- Requirements:

  (a) for all i, j: $i \neq j :: S_i \cap S_j \neq \Phi$

  (b) for all i: $i \in S_i$

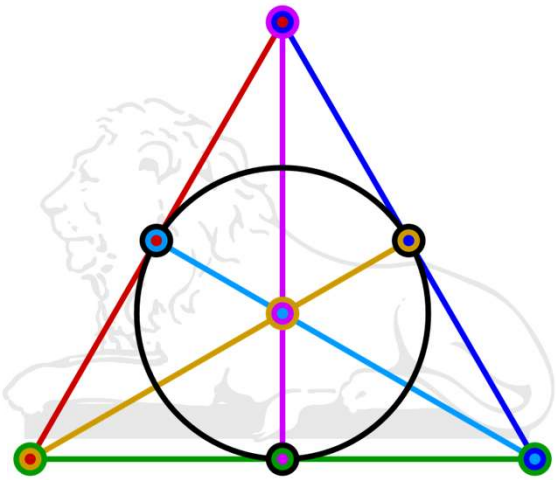
  (c) for all i: $|S_i| = K$, for some K

  (d) any node i is contained in exactly D Request Sets, for some D
- Finite projective plane of N points

Finite projective plane

- A finite projective plane of order n is formally defined as a set of $n^2 + n + 1$ points with the properties that:
  1. Any two points determine a line,
  2. Any two lines determine a point,
  3. Every point has $n + 1$ lines on it, and
  4. Every line contains $n + 1$ points.
- A finite projective plane exists when the order n is a power of a prime.

The Fano plane.



Finite projective plane of order $n = 2$

IIT ROORKEE

31

Quorum based Mutual Exclusion

- A site can have only one outstanding REPLY message at any time
- Lamport's / Ricart-Agarwala Algorithm?
- A site can send a REPLY only after it has received a RELEASE message for the previous REPLY

  => a site i locks all the sites in S_i in exclusive mode before executing its CS
- Intersection Property – ensures correctness
- Minimality Property – ensures efficiency
- Maekawa established that N = K(K-1) +1

- THE CHOICE OF S_i
  - The maximum number of subsets that satisfy property (a)
  - Set N equal to this maximum number so that K is minimized for a given N
- K = D = sqrt(N) must hold for Maekawa's algorithm
- The problem of finding a set of S_i's that satisfies these conditions is equivalent to **finding a finite projective plane of N points**
- There always exists a finite projective plane of order *k* if *k* is a power p^m, of a prime p

$S_1 = \{1, 2\}$
$S_3 = \{1, 3\}$
$S_2 = \{2, 3\}$

(a)

$S_1 = \{1, 2, 3\}$
$S_4 = \{1, 4, 5\}$
$S_6 = \{1, 6, 7\}$
$S_2 = \{2, 4, 6\}$
$S_5 = \{2, 5, 7\}$
$S_7 = \{3, 4, 7\}$
$S_3 = \{3, 5, 6\}$

(b)

Degenerated set of S_i's

- First create a symmetric set of S_i's for M where
  - (M - 1) is a power of a prime number and
  - M is the smallest integer larger than L.
- In this set of S_i's for K = M, each component is contained in M subsets
- Now replace each component greater than $N = L(L - 1) + 1$ in this set of S_i's by a number smaller than or equal to N
- This replacement is made by a different number each time.
- The resulting set of S_i's is not symmetric in the sense that the size of S_i is not always L.
- Degenerated sets can be created in the similar way if $N \neq K(K - 1) + 1$

- To obtain S_i's for N = 5, for instance, first create a set of S_i's for 7 and then replace 7 and 6 with 5 and 4, respectively, and remove S_7 and S_6

$$S_1 = \{1, 2, 3\}$$
$$S_4 = \{1, 4, 5\}$$
$$S_6 = \{1, 6, 7\}$$
$$S_2 = \{2, 4, 6\}$$
$$S_5 = \{2, 5, 7\}$$
$$S_7 = \{3, 4, 7\}$$
$$S_3 = \{3, 5, 6\}$$